

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

PART XVII

Advanced String Algorithms

Hashing, the prefix function, KMP, Z-algorithm, Trie, and an introduction to Aho-Corasick — pattern matching without ever re-scanning text you've already seen

PART XVII

Advanced String Algorithms

One theme, five techniques: never re-examine a character you don't have to

Every algorithm in this Part solves a version of the same question: given text and a pattern (or many patterns), find matches fast — faster than the $O(n \times m)$ brute force of trying every starting position and comparing character by character. Each technique gets there the same way: precompute something once, then reuse it to skip work a naive scan would repeat.

Contents of Part XVII

- 1. String Hashing (Rolling Hash)
- 2. The Prefix Function
- 3. KMP — Knuth-Morris-Pratt Pattern Matching
- 4. The Z-Algorithm
- 5. Trie (Prefix Tree)
- 6. Aho-Corasick — Multi-Pattern Matching (Introduction)
- Part XVII Revision, Cheat Sheet, and Interview Questions

Chapter 1 — String Hashing (Rolling Hash)

1.1 The problem in one line

Compare two substrings for equality in $O(1)$ instead of $O(\text{length})$ — by comparing numbers instead of characters.

Without hashing	With hashing
Comparing two length- k substrings costs $O(k)$ per comparison	Comparing two length- k substrings costs $O(1)$ once hashes are precomputed

1.2 The idea: treat a string as a number in some base

Map each character to a number, then read the string as a number in base p , reduced modulo a large prime m to keep the numbers small.

```
// hash(s) = (s[0]*p^0 + s[1]*p^1 + ... + s[n-1]*p^(n-1)) mod m
const long long P = 131;           // base, larger than the alphabet size
const long long M = 1e9 + 7;      // large prime modulus

long long stringHash(string s) {
    long long h = 0, pw = 1;
    for (char c : s) {
        h = (h + (c - 'a' + 1) * pw) % M;
        pw = (pw * P) % M;
    }
    return h;
}
```

1.3 Prefix hashes: the actual tool you'll use

Computing a fresh hash per substring is still $O(k)$. The real technique precomputes hashes of every prefix once, in $O(n)$, then answers any substring hash query in $O(1)$.

```

int n = s.size();
vector<long long> prefixHash(n + 1, 0), powP(n + 1, 1);
for (int i = 0; i < n; i++) {
    prefixHash[i + 1] = (prefixHash[i] + (s[i] - 'a' + 1) * powP[i]) %
    M;
    powP[i + 1] = (powP[i] * P) % M;
}

// hash of s[l..r] (0-indexed, inclusive), O(1):
long long getHash(int l, int r) {
    long long raw = (prefixHash[r + 1] - prefixHash[l] + M) % M;
    // dividing by p^l == multiplying by the modular inverse of
    p^l
    return (raw * modInverse(powP[l], M)) % M;
}

```

🔗 Why + M before the modulo

`prefixHash[r+1] - prefixHash[l]` can be negative in C++ (Part I's negative-modulo gotcha). Adding `M` before taking `% M` guarantees a non-negative result.

1.4 Dry run

String: "abcab", checking if `s[0..2]` ("abc") equals `s[?..?]` anywhere else — it doesn't here, but here's how the prefix array looks:

i	s[i]	prefixHash[i+1] (conceptually)
0	a	hash of "a"
1	b	hash of "ab"
2	c	hash of "abc"
3	a	hash of "abca"
4	b	hash of "abcab"

Any substring's hash is now a single subtraction + one multiplication away — no re-scanning of characters.

1.5 Collisions — the one real danger

Risk	Mitigation
Two different substrings hash to the same value (a collision)	Use a large prime modulus ($\approx 10^9$); on adversarial judges, use two different (p, m) pairs and compare the PAIR of hashes
An attacker crafts input to force collisions against a known modulus	Randomize the base p at runtime, or use double hashing

⚠ Single hashing can fail on adversarial test data

Codeforces problems have specifically targeted single-hash solutions with fixed, well-known (p, m) pairs. Double hashing (two independent hash functions, matched only if both agree) is the standard defense and is cheap to add.

Practice — Chapter 1

Problem 1.1 — Check if two substrings are equal [Level 2]

Given a string and multiple queries $(l1, r1, l2, r2)$, check if $s[l1..r1]$ equals $s[l2..r2]$ in $O(1)$ per query after preprocessing.

Hint 1: Precompute prefix hashes once; compare $\text{getHash}(l1, r1) == \text{getHash}(l2, r2)$.

Complexity: $O(n)$ preprocessing, $O(1)$ (or $O(\log M)$ with modular inverse) per query.

Problem 1.2 — Longest common substring length via hashing + binary search [Level 4]

Given two strings, find the length of their longest common substring.

Hint 1: Binary search on the answer length L (Part VI). For a fixed L , use rolling hashes to collect all length- L substring hashes of both strings into sets, and check for overlap in $O(n)$.

Key observation: Binary search on the answer only works because 'a common substring of length L exists' is monotonic — if true for L , it's

true for every smaller L too.

Complexity: $O(n \log n)$ with hashing + binary search, versus $O(n^2)$ or $O(n^3)$ brute force.

Chapter 2 — The Prefix Function

2.1 What it computes

In one sentence: for every prefix of a string, the prefix function tells you the length of the longest proper prefix of that prefix which is also a suffix of it.

“Proper” means not the whole substring itself. This single array, usually called `pi[]`, is the entire engine behind KMP (Chapter 3).

2.2 Worked example

String: "abcabcd"

i	prefix <code>s[0..i]</code>	<code>pi[i]</code>	why
0	a	0	no proper prefix exists
1	ab	0	"a" $\neq$ "b"
2	abc	0	no prefix of "abc" matches its suffix
3	abca	1	"a" (prefix) = "a" (suffix)
4	abcab	2	"ab" = "ab"
5	abcabc	3	"abc" = "abc"
6	abcabcd	0	no prefix of "abcabcd" matches its suffix

2.3 Computing it in $O(n)$

The naive approach checks every prefix against every suffix directly — $O(n^3)$, or $O(n^2)$ with some care. The linear algorithm reuses previously computed `pi` values instead of comparing from scratch.

```

vector<int> prefixFunction(string s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];                // length of best match
        while (j > 0 && s[i] != s[j])    // fall back using
            j = pi[j - 1];              previous answers
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

```

✓ Why this is $O(n)$, not $O(n^2)$

The inner while loop looks costly, but j only ever increases by at most 1 per outer iteration (the $j++$ line) and the while loop can only decrease it — so across the whole run, total decreases can't exceed total increases, which is bounded by n . Same amortized-analysis argument as the monotonic stack in Part IX.

Quick Revision

- $pi[i]$ = length of the longest proper prefix of $s[0..i]$ that's also a suffix of $s[0..i]$.
- Computed in $O(n)$ via amortized analysis — j only increases by 1 per step, and can only be reset by falling back through earlier pi values.
- This array is the direct foundation of KMP pattern matching (next chapter).

Chapter 3 — KMP: Knuth-Morris-Pratt Pattern Matching

3.1 The problem

Find every occurrence of a pattern string `pat` (length m) inside a text string `txt` (length n). Brute force: try every starting position, compare character by character — $O(n \times m)$. KMP: $O(n+m)$.

Approach	Time	Key idea
Brute force	$O(n \times m)$	On a mismatch, restart the pattern from its beginning at the next position
KMP	$O(n + m)$	On a mismatch, use the prefix function to skip directly to the next plausible alignment — never re-check characters already known to match

3.2 The trick

Build the prefix function of `pat + '#' + txt` (a separator character not in either string). Any position where the resulting `pi` value equals m (the pattern's length) marks a full match ending there.

```
vector<int> kmpSearch(string pat, string txt) {
    string combined = pat + "#" + txt;
    vector<int> pi = prefixFunction(combined);
    vector<int> matches;
    int m = pat.size();
    for (int i = m + 1; i < (int)combined.size(); i++) {
        if (pi[i] == m) {
            int startInTxt = i - 2 * m;      // convert index back
            to
            txt
            matches.push_back(startInTxt);
        }
    }
    return matches;
}
```

3.3 Dry run

pattern = "ab", text = "ababab". combined = "ab#ababab"

index in combined	char	pi value	pi == m (2)?
0	a	0	no
1	b	0	no
2	#	0	no
3	a	1	no
4	b	2	yes → match at txt index $4-2 \times 2=0$
5	a	1	no
6	b	2	yes → match at txt index $6-2 \times 2=2$
7	a	1	no
8	b	2	yes → match at txt index $8-2 \times 2=4$

Matches found at text positions 0, 2, 4 — correct: "ababab" contains "ab" starting at every even index.

The separator character matters

The separator must not appear in either string — otherwise a false match could bridge across pattern and text incorrectly, inflating the pi value beyond what a genuine match would produce. '#' is a common safe choice when working with lowercase letters.

Practice — Chapter 3

Problem 3.1 — Count pattern occurrences [Level 2]

Count how many times a pattern occurs in a text, including overlapping occurrences.

Hint 1: Run KMP as shown; overlapping matches are automatically included since pi doesn't skip past them.

Complexity: $O(n + m)$.

Problem 3.2 — Shortest string that contains a given string as prefix and suffix [Level 3]

Given a string s , find the shortest string formed by appending characters to s such that the result has some non-trivial prefix equal to its suffix (i.e., make it 'self-overlapping').

Hint 1: The prefix function of s itself tells you how much of s already overlaps with itself — $pi[n-1]$ gives the longest proper prefix that's also a suffix.

Key observation: The characters NOT covered by $pi[n-1]$ are exactly what's missing to complete the overlap — this connects the prefix function directly to string-construction problems, not just matching.

Complexity: $O(n)$.

Chapter 4 — The Z-Algorithm

4.1 What it computes

In one sentence: for every position i , $Z[i]$ is the length of the longest substring starting at i that matches a prefix of the whole string.

$Z[0]$ is conventionally left undefined or set to 0 (the whole string trivially matches itself, which isn't useful information).

4.2 Worked example

String: "aabxaab"

i	s[i..]	Z[i]	why
0	aabxaab	—	(undefined by convention)
1	abxaab	1	"a" matches prefix "a"; "ab" $\neq$ "aa"
2	bxaab	0	"b" $\neq$ "a"
3	xaab	0	"x" $\neq$ "a"
4	aab	3	"aab" matches the prefix "aab" exactly
5	ab	1	"a" matches; "ab" $\neq$ "aa"
6	b	0	"b" $\neq$ "a"

4.3 Computing it in $O(n)$

Maintain a window $[l, r]$ — the rightmost match against the prefix found so far — and reuse it to avoid redundant comparisons, conceptually similar to the amortized trick behind the prefix function.

```

vector<int> zFunction(string s) {
    int n = s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;                                // current best-known
    match window
    for (int i = 1; i < n; i++) {
        if (i < r)
            z[i] = min(r - i, z[i - l]);             // reuse previously
            computed info
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;                                   // extend by direct
            comparison
        if (i + z[i] > r) { l = i; r = i + z[i]; }    // update the
        window
    }
    return z;
}

```

Z-algorithm vs. KMP prefix function	
What it measures	Z: match against the WHOLE string's prefix. π : match between a prefix and a suffix of the SAME growing substring.
Typical use	Z: pattern matching via <code>s = pat + '#' + txt</code> , look for <code>Z[i] == len(pat)</code> . π : KMP matching, and self-overlap questions (border length).
Complexity	Both $O(n)$, both via amortized two-pointer / reuse arguments.

✓ Z-function pattern matching — same trick as KMP

Exactly like Chapter 3's combined string: build `pat + '#' + txt`, compute Z, and report a match at every position where `Z[i]` equals `len(pat)`. Many competitive programmers pick whichever of KMP or Z they remember better — both solve exact pattern matching in $O(n+m)$.

Practice — Chapter 4

Problem 4.1 — Count distinct substrings that are also prefixes**[Level 2]**

Given a string, count how many positions $i > 0$ have $Z[i] > 0$ (i.e., how many suffixes share a non-trivial prefix with the string).

Hint 1: Direct application: compute $Z[]$, count non-zero entries.

Complexity: $O(n)$.

Problem 4.2 — Find the shortest period of a string **[Level 3]**

Find the smallest k such that repeating a length- k substring (possibly truncating the last repetition) reproduces the whole string.

Hint 1: If $Z[k] == n - k$, then $s[0..k)$ repeated covers the rest of the string exactly — k is a valid period.

Hint 2: Find the smallest such k .

Key observation: This connects the Z-array to a structural property of the ENTIRE string, not just pairwise matching — periods show up again in Part XXII-style pattern problems.

Complexity: $O(n)$.

Chapter 5 — Trie (Prefix Tree)

5.1 What a trie is

In one sentence: a trie is a tree where each path from the root spells out a string, and strings sharing a common prefix share the same path for that prefix.

Operation	Complexity	Compare to a hash set of strings
Insert a string of length L	$O(L)$	$O(L)$ — similar
Search for an exact string of length L	$O(L)$	$O(L)$ average — similar
Check if any string has a given prefix	$O(L)$	$O(n \times L)$ — trie wins decisively
List all strings with a given prefix	$O(L + \text{output size})$	Not directly supported

✓ The trie's entire value proposition

A hash set answers “is this exact string present?” well, but cannot efficiently answer “does ANY stored string start with this prefix?” without scanning every entry. A trie answers that question in $O(\text{prefix length})$, independent of how many strings are stored.

5.2 Node structure and insertion

```
struct TrieNode {
    TrieNode* children[26] = {};    // nullptr by default; index =
    letter - 'a'
    bool isEndOfWord = false;
};

TrieNode* root = new TrieNode();

void insert(string word) {
    TrieNode* node = root;
    for (char c : word) {
        int idx = c - 'a';
        if (node->children[idx] == nullptr)
            node->children[idx] = new TrieNode();
        node = node->children[idx];
    }
    node->isEndOfWord = true;
}
```

5.3 Search and prefix check


```

bool search(string word) {                                // exact match
    TrieNode* node = root;
    for (char c : word) {
        int idx = c - 'a';
        if (node->children[idx] == nullptr) return false;
        node = node->children[idx];
    }
    return node->isEndOfWord;
}

bool startsWith(string prefix) {                          // any word with this prefix?
    TrieNode* node = root;
    for (char c : prefix) {
        int idx = c - 'a';
        if (node->children[idx] == nullptr) return false;
        node = node->children[idx];
    }
    return true;                                          // reached the end of the
    prefix path
}

```

⚠ **search() vs. startsWith() — one field is the whole difference**

Both functions walk the same path. `search` additionally requires `isEndOfWord` at the final node; `startsWith` doesn't care whether a word ends there, only whether the path exists at all. Confusing the two is the most common trie bug.

5.4 Dry run: inserting "cat", "car", "dog"

Insert	Path created/reused	isEndOfWord set at
cat	root→c→a→t (all new)	node "t"
car	root→c→a (reused) →r (new)	node "r"
dog	root→d→o→g (all new)	node "g"

"cat" and "car" share the path $\text{root} \rightarrow c \rightarrow a$ — this shared storage of common prefixes is the entire structural idea of a trie.

Practice — Chapter 5

Problem 5.1 — Count words with a given prefix [Level 2]

Extend the trie to support counting how many inserted words share a given prefix.

Hint 1: Add a counter field to each node, incremented every time `insert()` passes through it.

Approach: Add `int count = 0` to `TrieNode`, increment it at every node visited during insert (not just the final one).

Complexity: $O(L)$ per insert and per query.

Problem 5.2 — Maximum XOR of two numbers in an array [Level 4]

Given an array of non-negative integers, find the maximum XOR of any two elements, using a binary trie (each number's bits, MSB to LSB, form a path).

Hint 1: Build a trie of 0/1 bits (fixed depth ≈ 30 for 32-bit ints). For each number, greedily walk the trie trying to go the OPPOSITE bit direction at each level to maximize XOR.

Key observation: A binary trie applies the exact same shared-prefix idea to bit strings instead of character strings — the technique generalizes beyond lowercase-letter words.

Complexity: $O(n \times \text{bitlength})$, far better than the $O(n^2)$ brute-force pairwise check.

Chapter 6 — Aho-Corasick: Multi-Pattern Matching (Introduction)

6.1 The problem it solves

Given many patterns (not just one) and a single text, find every occurrence of every pattern — in one pass over the text.

Approach	Time	Notes
Run KMP once per pattern	$O(k \times (n + m))$ for k patterns	Re-scans the text k times
Aho-Corasick	$O(n + \sum m + \text{matches})$	Scans the text ONCE, for all patterns simultaneously

6.2 The core idea: a trie plus failure links

Aho-Corasick starts by inserting every pattern into a trie (Chapter 5) — that handles matching multiple patterns that share a common start. The genuinely new piece is a failure link at every node, pointing to the longest proper suffix of the current path that is also a prefix of some pattern in the trie. This is the Z/KMP idea (Chapters 2–4) generalized from one string to an entire trie of strings at once.

Concept	Single-pattern version (KMP)	Multi-pattern version (Aho-Corasick)
Structure walked	A single string	A trie of many strings
"Fallback" on mismatch	<code>pi[]</code> array	Failure links at every trie node
What a fallback points to	Longest prefix-that's-also-a-suffix of the CURRENT position	Longest suffix of the current path that is also SOME pattern's prefix, anywhere in the trie

6.3 Building the trie with pattern markers

```

struct ACNode {
    ACNode* children[26] = {};
    ACNode* fail = nullptr;           // failure link, built via BFS
    vector<int> patternIds;           // which pattern(s) end at thi
s
    node
};
ACNode* root = new ACNode();

void insert(string& pat, int id) {
    ACNode* node = root;
    for (char c : pat) {
        int idx = c - 'a';
        if (!node->children[idx]) node->children[idx] = new ACNode
();
        node = node->children[idx];
    }
    node->patternIds.push_back(id);
}

```

6.4 Building failure links with BFS

Failure links are built level by level (BFS, Part IX/XI's queue traversal) — a node's failure link depends on its parent's failure link already being computed, so processing must go top-down, level by level.

```

void buildFailureLinks() {
    queue<ACNode*> q;
    for (int c = 0; c < 26; c++) {
        if (root->children[c]) {
            root->children[c]->fail = root;
            q.push(root->children[c]);
        }
    }
    while (!q.empty()) {
        ACNode* cur = q.front(); q.pop();
        for (int c = 0; c < 26; c++) {
            ACNode* child = cur->children[c];
            if (!child) continue;
            ACNode* f = cur->fail;
            while (f != root && !f->children[c]) f = f->fail;
            child->fail = (f->children[c] && f->children[c] != child)
                ? f->children[c] : root;
            q.push(child);
        }
    }
}

```

🔑 **This is intentionally an introduction, not full mastery**

Aho-Corasick's failure-link construction and the subsequent text-scanning pass (which walks the automaton one character at a time, following failure links on mismatch exactly like KMP does) are dense. The goal here is recognizing the shape — trie + BFS-built failure links + single-pass scan — and knowing when a problem calls for it: “many patterns, one text, find all matches.”

📌 Quick Revision

- Trie stores many strings compactly by sharing common prefixes; $O(L)$ insert/search/prefix-check regardless of how many strings are stored.
- KMP's `pi[]` array = longest prefix-that's-also-a-suffix, computed in $O(n)$ via amortized reuse; powers $O(n+m)$ exact pattern matching.
- Z-function = longest match against the WHOLE string's prefix, starting at each position; also gives $O(n+m)$ pattern matching via the `pat+'#'+txt` trick.
- String hashing turns substring equality checks into $O(1)$ number comparisons after $O(n)$ preprocessing — watch for collisions on adversarial data.
- Aho-Corasick = trie + failure links (KMP's idea, generalized to a whole trie) — matches many patterns in one $O(n + \sum m)$ pass over the text.

Practice — Chapter 6

Problem 6.1 — Count total pattern occurrences [Level 3]

Given a dictionary of patterns and a text, count the total number of (possibly overlapping) occurrences of any pattern in the text, using Aho-Corasick conceptually.

Hint 1: Walk the text through the automaton one character at a time; whenever a node (or any node reachable by following failure links from it) has `patternIds`, count them.

Complexity: $O(n + \sum m + \text{total matches})$.

Part XVII — Master Revision

Which Technique Should I Use?

Situation	Technique
Compare substrings for equality repeatedly	String hashing (prefix hashes)
Find all occurrences of ONE pattern in a text	KMP or Z-algorithm (either works)
Find the longest prefix that's also a suffix of a growing string	Prefix function (π array)
Find where each suffix matches the string's own prefix	Z-function
Store many strings, check prefixes fast, autocomplete-style queries	Trie
Find all occurrences of MANY patterns in one text, in one pass	Aho-Corasick

Must Remember

Quick Revision

- All five techniques exist to avoid re-scanning text already examined — that's the unifying idea of this whole Part.
- Hashing: $O(1)$ substring comparison after $O(n)$ prefix-hash preprocessing; use double hashing against adversarial data.
- $\pi[i]$ (prefix function): longest proper prefix of $s[0..i]$ that's also its suffix; computed in $O(n)$.
- KMP: build π on $\text{pat} + \# + \text{txt}$; report a match wherever $\pi[i] == \text{len}(\text{pat})$. $O(n+m)$.
- $Z[i]$: longest match between $s[i..]$ and the whole string's prefix; also gives $O(n+m)$ pattern matching.
- Trie: $O(L)$ insert/search/prefix-check, independent of how many strings are stored — its entire value is fast prefix queries.
- Aho-Corasick: a trie with BFS-built failure links, generalizing KMP's single-string fallback to many patterns at once.

Common Mistakes Recap

Mistake	Fix
Using a single hash with a small/well-known modulus	Use a large prime ($\approx 10^9$) and consider double hashing
Forgetting $+M$ before $\%M$ when subtracting hash values	C++'s $\%$ can return negative — always normalize with $+M$
Choosing a separator character that can appear in the strings	Use a character guaranteed absent from the alphabet (e.g. '#')
Confusing trie's <code>search()</code> (needs <code>isEndOfWord</code>) with <code>startsWith()</code> (doesn't)	Check which one the problem actually needs before coding
Building Aho-Corasick failure links in the wrong order (not BFS/level order)	A node's failure link needs its parent's failure link already built — must process level by level

Complexity Table

Technique	Preprocessing	Per-query / matching
String hashing	$O(n)$	$O(1)$ per substring comparison
Prefix function (π)	$O(n)$	—
KMP matching	$O(n + m)$	included in preprocessing
Z-function	$O(n)$	—
Trie insert/search/prefix-check	$O(L)$ per string inserted	$O(L)$ per query
Aho-Corasick (build + scan)	$O(\sum m)$	$O(n + \text{matches})$ to scan text

Interview-Style Questions

- Why is string hashing vulnerable on adversarial test data, and how does double hashing mitigate this?
- Explain why the prefix function can be computed in $O(n)$ despite an inner while loop that appears to add extra iterations.
- How does KMP use the prefix function to avoid re-examining characters of the text after a mismatch?
- What is the key difference between what the Z-function measures and what the KMP prefix function measures?
- Why is a trie better than a hash set specifically for prefix queries, even though both can do exact-match lookups in similar time?
- How does Aho-Corasick's failure link generalize the idea behind KMP's prefix function?

Cheat Sheet — Part XVII

```
// Prefix hashing
long long h=0, pw=1;
for (char c : s) { h=(h+(c-'a'+1)*pw)%M; pw=(pw*P)%M; }

// Prefix function (KMP core)
vector<int> pi(n,0);
for (int i=1;i<n;i++){
    int j=pi[i-1];
    while(j>0 && s[i]!=s[j]) j=pi[j-1];
    if (s[i]==s[j]) j++;
    pi[i]=j;
}

// KMP matching: build pi on pat+"#"+txt, report i where
pi[i]==pat.size()
```

```

// Z-function
vector<int> z(n,0); int l=0,r=0;
for (int i=1;i<n;i++){
    if (i<r) z[i]=min(r-i, z[i-l]);
    while (i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
    if (i+z[i]>r){ l=i; r=i+z[i]; }
}

// Trie
struct TrieNode { TrieNode* ch[26]={}; bool end=false; };
void insert(TrieNode* root, string w){
    TrieNode* n=root;
    for (char c : w) {
        int i=c-'a';
        if(!n->ch[i]) n->ch[i]=new TrieNode();
        n=n->ch[i];
    }
    n->end=true;
}

```

Coming up in Part XVIII

Part XVIII — Advanced Data Structures covers the Fenwick Tree, Segment Tree (with lazy propagation), Sparse Table, and Binary Lifting / LCA — structures that answer range queries (sum, minimum, etc.) over an array in $O(\log n)$ per query, the same way this Part's structures answered questions over strings efficiently.